First 3D Scene: Technical Report

Nicolas Conde Salazar

Introduction

This report documents the computer graphics techniques implemented in *First 3D Scene*, a Unity-based 3D environment featuring a player-controlled aircraft, an AI seeker agent, and a simple obstacle course. The implementation demonstrates core concepts including transforms, physics, steering behaviors, camera control, materials, and textures.

Transforms and Scene Layout

Transforms define position, rotation, and scale using 4×4 transformation matrices. Player movement is implemented using Unity's transform system to enable direct manipulation in world space.

Implementation

PlayerMover.cs implements WASD movement via `transform.Translate(movement * speed * Time.deltaTime, Space.World)` and Q/E rotation via `transform.Rotate()`.

PropellerSpin.cs applies continuous local rotation using `transform.Rotate(0, spinSpeed * Time.deltaTime, 0, Space.Self)`, where the propeller is a child object inheriting parent transforms.

Obstacles consist of eight cube primitives positioned with calculated Y-values to align with the ground plane.

Rationale

Transform-based movement provides immediate, responsive control while avoiding unnecessary physics overhead.

Physics and Collision

Unity's Rigidbody physics system is used for collision detection between the AI seeker and static obstacles. Collisions occur when primitive colliders intersect.

Implementation

SeekSteering.cs uses `Rigidbody.MovePosition()` inside `FixedUpdate()` for physics-consistent motion. The Rigidbody is configured with `CollisionDetectionMode.Continuous` and constrained with `FreezePositionY`. Obstacles use `BoxCollider` components, while the seeker uses a `SphereCollider`.

Rationale

Using Rigidbody physics allows Unity to handle collision response automatically, eliminating the need for manual intersection checks.

Steering Behaviors

The seeker uses Craig Reynolds' Seek steering behavior, which computes a steering force toward a target position:

```
steering = (normalize(target - position) * maxSpeed) - currentVelocity
```

An arrival modifier reduces speed as the agent approaches the target:

```
speed = maxSpeed * (distance / arriveDistance)
```

Implementation

SeekSteering.cs computes desired velocity toward the player, derives steering as the difference from current velocity, applies arrival behavior near the target, and updates orientation using `Quaternion.LookRotation()`.

Rationale

Steering behaviors produce smooth, natural-looking pursuit motion without requiring complex pathfinding algorithms.

Camera

A third-person camera smoothly follows the player to maintain spatial awareness and visual comfort.

Implementation

ThirdPersonFollow.cs maintains a positional offset from the player, interpolates motion with `Vector3.SmoothDamp(current, desired, ref velocity, smoothTime)`, and orients using `transform.LookAt(target + lookAtOffset)`.

Rationale

Smooth damping provides gradual acceleration and deceleration, preventing abrupt camera movements.

Materials and Colors

Materials define surface appearance, while runtime instancing allows each object to maintain unique visual properties.

Implementation

PatternEffect.cs creates a per-object material instance using `new Material(renderer.sharedMaterial)` and sets a base tint via `material.SetColor("`$_{BaseColor}$`", tintColor)`. Obstacles use varying gray tones (approximately RGB 0.15–0.55) to improve differentiation.

Rationale

Material instancing prevents unintended changes to shared assets while enabling per-object customization.

Textures

Textures map 2D images onto 3D geometry. Tiling controls how frequently a texture repeats across a surface.

Implementation

GridEffect.cs applies a hexagonal texture pattern to the ground plane, computes tiling based on object scale (e.g., `tilingValue = tiling * scale / 10`), and assigns the texture via `material.SetTexture("`$_{BaseMap}$`", patternTexture)`.

Rationale

Scale-aware tiling ensures consistent texture density regardless of object size.

References

1. Reynolds, C. W. (1999). *Steering Behaviors for Autonomous Characters*. Game Developers Conference. https://www.red3d.com/cwr/papers/1999/gdc99steer.html
2. Unity Technologies. (2024). *Unity Manual: Rigidbody*. https://docs.unity3d.com/Manual/class-Rigidbody.html
3. Unity Technologies. (2024). *Unity Scripting API: Transform*. https://docs.unity3d.com/ScriptReference/Transform.html
4. Unity Technologies. (2024). *Unity Scripting API: Vector3.SmoothDamp*. https://docs.unity3d.com/ScriptReference/Vector3.SmoothDamp.html
5. Unity Technologies. (2024). *Unity Scripting API: Material*. https://docs.unity3d.com/ScriptReference/Material.html

Game Image Reference